

Finding Missing Input Validation in TEEs via LLM-Assisted Symbolic Execution

Chengyan Ma
Singapore Management University
Singapore

Jieke Shi
Singapore Management University
Singapore

Ruidong Han
Singapore Management University
Singapore

Ye Liu
Singapore Management University
Singapore

Yuqing Niu
Singapore Management University
Singapore

David Lo
Singapore Management University
Singapore

Abstract

Trusted Execution Environments (TEEs) provide hardware-enforced isolation that protects sensitive code and data from untrusted software. Despite their strong security guarantees, analyzing TEE applications remains challenging due to the high cost and complexity of configuring complete TEE build and runtime environments, as well as the limited observability imposed by hardware isolation. This paper presents SymTEE, a novel large language model (LLM)-assisted symbolic execution framework for detecting missing input validation issues in TEE applications without requiring real TEE setups. SymTEE begins by leveraging Abstract Syntax Tree (AST) analysis to extract TEE code slices that may lack sufficient input validation, and then employs an LLM (GPT-5 in our case) to automatically convert the extracted slices into KLEE-compatible harness programs containing lightweight mock execution environments for symbolic analysis. Evaluations on 26 vulnerabilities (11 real-world and 15 synthetic) show that SymTEE achieves 100% precision and 92.3% recall in detecting missing input validation vulnerabilities while incurring an average analysis cost of only \$0.05. These results demonstrate the effectiveness and practicality of SymTEE's pioneering paradigm of LLM-assisted symbolic execution, where LLMs autonomously generate mock environments to enable automated security analysis without complex setup, providing a more accessible and scalable framework for trusted computing systems.

ACM Reference Format:

Chengyan Ma, Jieke Shi, Ruidong Han, Ye Liu, Yuqing Niu, and David Lo. 2026. Finding Missing Input Validation in TEEs via LLM-Assisted Symbolic Execution. In *AI Foundation Models and Software Engineering (FORGE'26)*, April 12–13, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 6 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

A Trusted Execution Environment (TEE) is a segregated area within a processor and its memory that isolates sensitive code and data from potentially compromised software or operating systems [21,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FORGE'26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

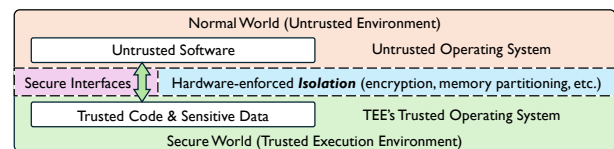


Figure 1: Architecture of a Trusted Execution Environment (TEE), which is isolated from the normal world and can only be accessed via secure APIs.

```
1 // GitHub Project: shuaifengyun/basicAlg_use (commit 327f23d)
2 // File: core/tee/crypto_ta_pbkdf2.c
3 -261,6 +261,9 void g_CryptoTaPbkdf_PBKDF2(..., int dkLen)
4 // Improper TEE usage: missing input validation of data from untrusted
5 // normal world
6 - TEE_MemMove(output, resultBuf, dkLen);
7 // Fix: Added input validation to prevent buffer overflow
8 + if (dkLen > 512)
9 +     return TEE_ERROR_BAD_PARAMETERS;
10 + TEE_MemMove(output, resultBuf, dkLen);
11 return TEE_SUCCESS;
```

Figure 2: Example of missing input length validation in a memory copy operation from the untrusted normal world to a fixed-size buffer within the TEE, found in a cryptographic library on GitHub (shuaifengyun/basicAlg_use). Our SymTEE successfully detected this issue and suggested a fix.

24, 33], thereby ensuring the confidentiality and integrity of critical computations and assets like cryptographic keys [3] and biometric information [2, 21]. Major hardware vendors, including Intel and ARM, have developed TEE infrastructures, with Intel Software Guard Extensions (SGX) [11] and ARM TrustZone [32] being two widely-adopted implementations. As shown in Figure 1, they typically divide the computing environment into a normal world and a secure world (TEE). The normal world hosts untrusted components that cannot access or tamper with data in the secure world, which is protected by hardware-enforced isolation mechanisms like memory partitioning and encryption, and can only be accessed via secure application programming interfaces (APIs) provided by hardware vendors. TEEs have become a cornerstone of trustworthy computing, supporting applications such as mobile payments [1, 23, 36], digital identity management [13, 24], and many other security-critical services that collectively safeguard assets and user data.

However, developing secure TEE-based applications is challenging. Similar to the insecure practices often observed in cryptographic and other security-critical contexts [18, 29, 30], many developers with limited security expertise may misuse or insecurely apply the secure APIs provided by hardware vendors when building TEE applications, thereby undermining security guarantees or even introducing new attack surfaces. A common class of vulnerabilities in TEE applications arises from insufficient validation and sanitization of input data received from the potentially compromised normal world, including improper or missing checks on data size, format, or validity [26]. Such flaws can lead to severe memory corruption issues such as buffer, stack, and heap overflows [22, 26]. Figure 2 illustrates this issue with a vulnerability found in a TEE-based cryptographic library on GitHub, where the developer failed to validate the input length of parameters received from the normal world. This omission can result in buffer overflows, turning the TEE from a security boundary into an exploitable target, where attackers can craft malicious inputs that corrupt memory and potentially compromise the secure environment.

Detecting missing input validation issues is challenging because such bugs often occur only under specific input conditions and execution paths (e.g., when input sizes exceed buffer limits), making them difficult for static analysis to reliably confirm [22, 26] and often requiring dynamic analysis to capture their runtime semantics. However, the unique architecture of TEEs introduces multiple obstacles for dynamic analysis techniques such as fuzzing and symbolic execution, which have proven effective for conventional software [4, 27]. Concretely, TEEs are accessible only through vendor-defined secure interfaces that common security-oriented fuzzers [28] or symbolic executors [7] do not natively support [21, 26, 31], forcing researchers to build custom harnesses and loaders that are often tedious and error-prone. Executing real TEEs further requires a specialized toolchain and environment for cross-compilation and signing on hardware with TEE support [24], so even running a simple “hello-world” test case demands substantial setup effort and may be infeasible when appropriate hardware is unavailable. Moreover, TEE isolation prevents the normal world from observing or instrumenting the secure world, rendering standard debugging methods ineffective because crashes or coverage data cannot be collected without modifying the TEE or its APIs. Collectively, these constraints cause most off-the-shelf dynamic analysis tools to fail when analyzing TEE applications, thereby overlooking subtle input-validation flaws hidden within the secure world.

In this paper, we present SymTEE, a Large Language Model (LLM)-assisted symbolic execution framework for detecting missing input validation issues in TEE applications. The core idea of SymTEE is to tackle the aforementioned challenges by leveraging LLMs to automatically generate mock environments that emulate TEE runtimes with minimal stub dependencies, eliminating the need for full TEE setups or specialized hardware.

SymTEE starts with performing Abstract Syntax Tree (AST)-based analysis to extract code slices that may lack input validation, enabling focused analysis on high-risk code regions. Then, it uses an LLM (OpenAI GPT-5 in our case) to expand each code slice into a complete program with minimal stubs compatible with KLEE [7], a dynamic symbolic execution engine, allowing symbolic execution to run locally without instrumenting a real TEE. Finally, SymTEE

applies KLEE to explore execution paths and identify inputs that violate the assertion oracles generated in the previous step by the LLM, which encode input validation checks.

We evaluate SymTEE on a benchmark of 26 vulnerabilities, including 11 real-world cases from GitHub projects and 15 synthetic ones from the recent benchmark PartitioningE-Bench [26]. SymTEE achieves a precision of 100% and a recall of 92.3% in detecting missing input validation issues, with each analysis requiring, on average, 5,931 tokens (costing approximately 0.05 USD). These results demonstrate that SymTEE is both effective and practical for detecting TEE vulnerabilities and represent an early yet promising step toward LLM-assisted symbolic execution based on automatically generated mock environments, paving the way for more accessible and efficient security analysis in trusted computing systems and other constrained environments.

Our contributions:

- We propose the first LLM-assisted symbolic execution approach for detecting missing input validation issues in TEE applications, without requiring complex runtime setups or hardware support.
- We perform a comprehensive evaluation on 26 vulnerabilities, including both real-world and synthetic programs, demonstrating that SymTEE achieves 100% precision and 92.3% recall.
- As an emerging line of work, we provide insights for future research and outline potential extensions, like enhancing SymTEE with specialized LLMs and broadening its vulnerability coverage.

2 Preliminaries and Related Work

2.1 Preliminaries

Trusted Execution Environments (TEEs). As introduced in Section 1, TEEs are isolated execution regions within a processor that protect sensitive code and data from untrusted software. This isolation is enforced by hardware mechanisms, making program analysis and testing of TEE applications particularly challenging because internal states and runtime behaviors are largely inaccessible to external tools. Moreover, setting up a TEE environment also requires complex configurations such as cross-compilation, secure boot, and binary signing with vendor-specific toolchains [16, 24]. These factors collectively hinder the use of traditional dynamic analysis techniques such as fuzzing and symbolic execution for vulnerability detection in TEE applications, highlighting the need for alternative approaches like SymTEE that can perform analysis locally without relying on actual TEE hardware or runtime environments.

Symbolic Execution. Symbolic execution [4, 7, 10] is a program analysis technique that treats inputs as symbolic variables instead of concrete values, enabling simultaneous exploration of multiple execution paths. For each path, it generates logical constraints that describe the conditions under which that path is taken. By solving these constraints with Satisfiability Modulo Theories (SMT) solvers [12], concrete inputs can be derived that trigger specific program behaviors, facilitating targeted testing and vulnerability discovery. Tools such as KLEE [7], S2E [10], and angr [35] have been widely adopted in security analysis to identify bugs and verify correctness. In our work, since TEE applications are primarily written in C/C++, we use KLEE, one of the most widely used symbolic execution engines for C/C++ programs, to systematically explore code paths and detect missing input validation vulnerabilities.

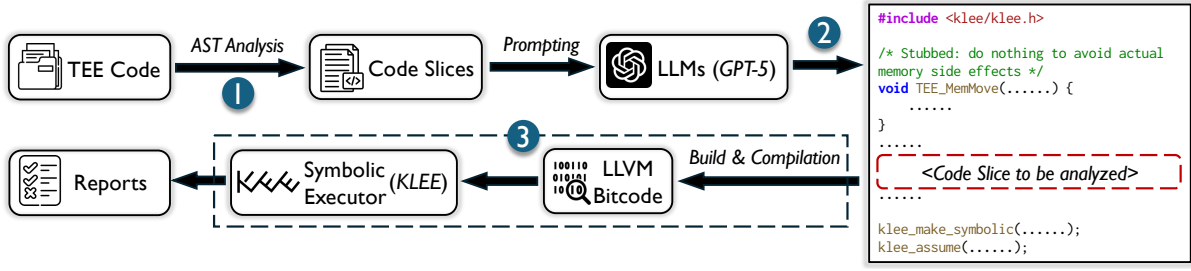


Figure 3: Overview of the SymTEE workflow consisting of three stages: ❶ AST analysis for extracting code slices; ❷ LLM-assisted generation of mock environments; and ❸ dynamic symbolic execution for vulnerability detection.

2.2 Related Work

Recent research has explored methods for identifying vulnerabilities in TEE applications. DITING [26] is a static analyzer tailored for TEE projects that tracks data flow between the normal world and the TEE to detect insecure patterns such as unencrypted data exchange. Other efforts [8, 16, 17] have empirically studied and characterized common TEE weaknesses, offering valuable guidance for rule-based static analysis. Beyond static inspection, dynamic analysis has also been investigated. COIN [22] introduces a concolic execution framework for testing Intel SGX, while PARTEMU [19] employs QEMU [5] to emulate TrustZone for dynamic testing. Fuzzing-based approaches such as TEEzz [6], EnclaveFuzz [9], and TEEFuzzer [14] aim to generate valid inputs and improve code coverage through techniques like input format inference and coverage-guided mutation. However, all existing dynamic methods require dedicated TEE compilation and runtime environments, inevitably facing challenges of complex setup and reliance on TEE-enabled hardware.

3 Methodology

As illustrated in Figure 3, SymTEE contains 3 steps: ❶ Analyzing the TEE code via its AST to extract all the code slices that may have vulnerabilities. ❷ We use LLM to expand the code slice into complete code that can be analyzed by KLEE (a dynamic symbolic execution engine). ❸ After compiling the code to LLVM bytecode, we can use dynamic symbolic execution to analyze whether a vulnerability actually exists. We elaborate on these steps as follows.

The first step performs AST analysis to identify code slices that may lack input validation. Since TEE memory copy operations must use fixed APIs provided by vendors (e.g., `TEE_Move(dest, src, len)`), SymTEE detects such operations and tracks data flow through the AST to locate the field that specifies the size of the destination buffer `dest`. It then checks whether this size field is compared against the copy length (`len`) to ensure proper boundary validation. If no such comparison is found, the operation is flagged as potentially vulnerable to buffer overflow. At this stage, SymTEE extracts the entire function containing the suspicious operation as a code slice for subsequent dynamic analysis.

Because the extracted TEE code slices depend on vendor libraries and hardware, they cannot be compiled or executed on a normal host directly. In Step ❷, we therefore leverage an LLM to generate a mock environment for each slice. The mock environment has

two goals. First, it supplies minimal stub implementations for external dependencies (e.g., `TEE_*` APIs and types such as `TEE_Param`) so the slice compiles as a standalone C program. Second, it constructs a KLEE-compatible security harness (`int main(void)`) for dynamic analysis: the harness marks attacker-controlled inputs (for example, buffer length `len` or data content) symbolic via `klee_make_symbolic` and constrains them with `klee_assume`. It also embeds a security oracle (using `klee_assert`) that checks function return values or an instrumentation flag (e.g., `g_checked`) to determine whether the code correctly rejects malicious inputs before performing memory operations. The result is a single, self-contained C file suitable for compilation to LLVM bytecode and subsequent symbolic execution in Step ❸. Figure 4 illustrates a KLEE-compatible program synthesized from a vulnerable TEE slice: the LLM embeds the original `produce` function (with the unsafe `TEE_MemMove`), generates minimal type and API stubs so the slice compiles, and synthesizes a `main` harness that makes `attacker_buf` and `attacker_size` symbolic (constrained with `klee_assume`). A `klee_assert` oracle flags when `g_checked` is unset and the condition `attacker_size > 512UL` is met, allowing KLEE to find concrete inputs that violate the check.

Once the LLM-generated KLEE harness is produced, we compile it to LLVM bytecode and pass it to KLEE in Step ❸. KLEE explores execution paths driven by the symbolic inputs (e.g., `attacker_size`, `attacker_buf` in the example) and searches for concrete values that violate the `klee_assert` oracle. If KLEE successfully finds a path that triggers the assertion (i.e., a path where inputs of invalid size, range, or format are processed without proper validation, leading to memory corruption), the slice is reported as a true vulnerability. SymTEE then generates a report describing the execution path, the concrete inputs, and the location of the missing validation.

4 Evaluation

We evaluate SymTEE on a benchmark comprising 26 memory operations without input validation. This benchmark is constructed from two distinct sources:

- **Synthetic Test Cases:** 15 unsafe memory operation vulnerabilities chosen from the PartitioningE-Bench test suite [26], a collection of security flaws originally created by manual injection into the function bodies with diverse coding styles.
- **Real-World Projects:** 11 real-world vulnerable code fragments collected from open-source TEE projects on GitHub (*optee-sdp* and *basicAlg_use*).

```

1 #include <klee/klee.h>
2
3 typedef struct { void* buffer; unsigned long size; } memref_t;
4 typedef struct { memref_t memref; } TEE_Param;
5 volatile int g_checked = 0;
6
7 void TEE_MemMove(void* dest, const void* src, size_t n) {
8     (void)dest; (void)src; (void)n;
9     /* Stubbed: do nothing to avoid actual memory side effects */
10 }
11
12 void produce(TEE_Param params[4]) {
13     char str[512];
14     TEE_MemMove(str, params[0].memref.buffer, params[0].memref.size);
15 }
16
17 int main(void) {
18     TEE_Param params[4];
19
20     char buf[4096];
21     unsigned long size;
22     klee_make_symbolic(&size, sizeof(size), "size");
23     klee_assume(size <= 4096UL);
24
25     params[0].memref.buffer = buf;
26     params[0].memref.size = size;
27
28     produce(params);
29
30     if (size > 512UL) {
31         klee_assert(g_checked && "Missing input validation");
32     }
33     .....
34 }

```

Figure 4: An example of LLM-generated KLEE harness.

Table 1: Effectiveness evaluation and GPT-5 token usage of SymTEE. Vul is the number of issues, N is the number of detection results, and TP is the number of true positives. P and R indicates the precision and recall, respectively.

Project	#Vul	Detection Results (N/TP)	Token Usage (On Avg.)	Token Cost (On Avg.)
Result on PartitioningE-Bench [26]				
PartitioningE-Bench	15	13/13	4,223	\$0.03
Result on real-world projects				
optee-sdp	9	9/9	7,215	\$0.06
basicAlg_use	2	2/2	6,354	\$0.05
Total	26	24/24	5,931	\$0.05
P(%) / R(%)		100/92.3		

* P(%) = #TP / #N, R(%) = #TP / #Vul.

By incorporating both complex vulnerabilities sourced from real-world projects and synthetic flaws from a controlled test suite, the evaluation of SymTEE ensures broad coverage and high representativeness. SymTEE is implemented in Python with GPT-5 and evaluated on a server running Ubuntu 24.04, equipped with a 48-core 2.3 GHz AMD EPYC 7643 processor and 512 GB RAM.

Table 1 presents the results. Out of the 26 known vulnerabilities in the benchmark, SymTEE successfully detected 24 issues, achieving a recall of 92.3% and a precision of 100%. The two false negatives were primarily due to the limitations of the initial static analysis phase (❶), which failed to extract the relevant code slices containing the vulnerable memory operation. For all 24 code slices that were successfully extracted by the static analysis, the subsequent dynamic symbolic execution phase (❷) correctly identified

the missing input validation vulnerability. We also measured the token usage and cost of GPT-5. On average, analyzing each vulnerability required about 5,931 tokens, resulting in an average token cost of \$0.05 per vulnerability. Overall, these results demonstrate that SymTEE can reliably generate compilable mock environments and KLEE harnesses, accurately identify unsafe memory operations through symbolic execution, and achieve strong cost-effectiveness in TEE vulnerability detection.

5 Future Plans

Strengthening AST Analysis and LLM Specialization. Our failure-case analysis (Section 4) suggests that SymTEE could benefit from a more precise AST-based slicer to better capture potentially vulnerable code slices. Future work can improve the slicing heuristics and integrate interprocedural data-flow analysis to enhance the slicer’s accuracy. Moreover, the effectiveness of SymTEE also depends on the LLM’s ability to generate correct mock environments and KLEE harnesses. Although general-purpose models performed well on our benchmark, they may struggle with real-world TEE applications that involve complex APIs or uncommon features. Future research can investigate training or fine-tuning LLMs on TEE codebases to strengthen their understanding of TEE semantics and enable the generation of more reliable mock environments.

Benchmark and Scope Extensions. Currently, SymTEE specializes in detecting missing input validation vulnerabilities in TEE applications. However, TEEs also suffer from other security issues, such as improper cryptographic usage [8, 26]. We plan to extend the current benchmark to include more diverse vulnerabilities and real-world TEE projects, allowing a broader and more realistic evaluation of SymTEE’s versatility. Moreover, our proposed paradigm of LLM-assisted symbolic execution can be generalized to other domains, such as embedded systems [15, 37], which also involve complex toolchains and hardware-reliant debugging processes. We aim to adapt SymTEE to these environments to make security analysis more accessible across a wider range of critical systems.

Trust and Synergy with Developers. Automated vulnerability detection assisted by LLMs, including our approach, currently involves limited interaction with developers, which poses challenges for building trust and achieving effective collaboration. This issue has been increasingly discussed in the context of AI-assisted software engineering [20, 25, 34]. Future research should explore mechanisms to promote closer collaboration, for example, by clarifying the impact of mocked environments, and allowing developers to review or refine them for safer execution. Strengthening this trust and interaction will help LLM-powered symbolic executors evolve into reliable collaborators, aligning with the vision of trustworthy and synergistic AI in software engineering [25].

6 Conclusion

This paper presents SymTEE, a novel LLM-assisted symbolic execution approach for detecting missing input validation issues in TEE applications. SymTEE leverages LLMs to automatically generate mock environments and KLEE-compatible security harnesses, eliminating the need for complex TEE runtime setups or specialized hardware. Experiments on 11 real-world vulnerabilities and 15 synthetic cases show that SymTEE achieves 100% precision and

92.3% recall at an average cost of about \$0.05 per analysis, demonstrating both effectiveness and practicality in uncovering input validation flaws. Our study also points to opportunities for extending LLM-assisted symbolic execution to broader classes of TEE vulnerabilities and other constrained environments like embedded systems. As an emerging result with substantial potential impact, SymTEE embodies an early yet promising paradigm where LLMs autonomously generate mock environments to enable automated security analysis without complex setup, paving the way for more accessible and scalable analysis for trusted computing systems.

Acknowledgments

This research/project is supported by the National Research Foundation, Singapore, and the Cyber Security Agency of Singapore under its National Cybersecurity R&D Programme (Proposal ID: NCR25-DeSCEmT-SMU). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore, and the Cyber Security Agency of Singapore.

Data Availability

An anonymized replication package, including the implementation of SymTEE and detailed instructions for running it, is available at: <https://github.com/CharlieMCY/SymTEE>.

References

- [1] Waqas Ahmed, Aamir Rasool, Abdul Rehman Javed, Neeraj Kumar, Thippa Reddy Gadekallu, Zunera Jalil, and Natalia Kryvinska. 2021. Security in Next Generation Mobile Payment Systems: A Comprehensive Survey. *IEEE Access* 9 (2021), 115932–115950. doi:10.1109/ACCESS.2021.3105450
- [2] Sunil Anasuri. 2023. Confidential Computing Using Trusted Execution Environments. *International Journal of AI, BigData, Computational and Management Studies* 4, 2 (Jun. 2023), 97–110. doi:10.63282/3050-9416.IJAIDCMS-V4I2P111
- [3] Matthew Areno and Jim Plusquellic. 2012. Securing Trusted Execution Environments with PUF Generated Secret Keys. In *2012 IEEE 11th International Conference on Trust, Security and Privacy in Computing and Communications*. 1188–1193. doi:10.1109/TrustCom.2012.255
- [4] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. *ACM Comput. Surv.* 51, 3, Article 50 (May 2018), 39 pages. doi:10.1145/3182657
- [5] Fabrice Bellard. 2005. QEMU, a fast and portable dynamic translator. In *Proceedings of the Annual Conference on USENIX Annual Technical Conference* (Anaheim, CA) (ATEC '05). USENIX Association, USA, 41.
- [6] Marcel Busch, Aravind Machiry, Chad Spensky, Giovanni Vigna, Christopher Kruegel, and Mathias Payer. 2023. TEEzz: Fuzzing Trusted Applications on COTS Android Devices. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1204–1219.
- [7] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (San Diego, California) (OSDI'08). USENIX Association, USA, 209–224.
- [8] David Cerdeira, Nuno Santos, Pedro Fonseca, and Sandro Pinto. 2020. SoK: Understanding the Prevailing Security Vulnerabilities in TrustZone-assisted TEE Systems. In *2020 IEEE Symposium on Security and Privacy (SP)*. 1416–1432.
- [9] Liheng Chen, Zheming Li, Zheyu Ma, Yuan Li, Baojian Chen, and Chao Zhang. 2024. EnclaveFuzz: Finding Vulnerabilities in SGX Applications. In *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society.
- [10] Vitaly Chipounov, Volodymyr Kuznetsov, and George Candea. 2012. The S2E Platform: Design, Implementation, and Applications. *ACM Trans. Comput. Syst.* 30, 1, Article 2 (Feb. 2012), 49 pages. doi:10.1145/2110356.2110358
- [11] Intel Corporation. 2025. Intel® Software Guard Extensions (Intel® SGX). <https://www.intel.com/content/www/us/en/products/docs/accelerator-engines/software-guard-extensions.html>. [Accessed 05-11-2025].
- [12] Leonardo De Moura and Nikolaj Bjørner. 2011. Satisfiability modulo theories: introduction and applications. *Commun. ACM* 54, 9 (Sept. 2011), 69–77. doi:10.1145/1995376.1995394
- [13] Amit Dua, Siddharth Sekhar Barpanda, Neeraj Kumar, and Sudeep Tanwar. 2020. Trustful: A Decentralized Public Key Infrastructure and Identity Management System. In *2020 IEEE Globecom Workshops (GC Wkshps)*. 1–6. doi:10.1109/GCWkshps50303.2020.9367444
- [14] Guoyun Duan, Yuanzhi Fu, Boyang Zhang, Peiyao Deng, Jianhua Sun, Hao Chen, and Zhiwen Chen. 2023. TEEFuzzer: A fuzzing framework for trusted execution environments with heuristic seed mutation. *Future Gener. Comput. Syst.* 144 (2023), 192–204.
- [15] Zachary Enghardt, Richard Li, Dilini Nissanka, Zhihan Zhang, Girish Narayanswamy, Joseph Breda, Xin Liu, Shwetak Patel, and Vikram Iyer. 2024. Exploring and Characterizing Large Language Models for Embedded System Development and Debugging. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI EA '24). Association for Computing Machinery, New York, NY, USA, Article 150, 9 pages. doi:10.1145/3613905.3650764
- [16] Shufan Fei, Zheng Yan, Wenxiu Ding, and Haomeng Xie. 2021. Security Vulnerabilities of SGX and Countermeasures: A Survey. *ACM Comput. Surv.* 54, 6, Article 126 (July 2021), 36 pages.
- [17] Fabian Fleischer, Marcel Busch, and Phillip Kuhrt. 2020. Memory corruption attacks within Android TEEs: a case study based on OP-TEE. In *Proceedings of the 15th International Conference on Availability, Reliability and Security* (Virtual Event, Ireland) (ARES '20). Association for Computing Machinery, New York, NY, USA, Article 53, 9 pages.
- [18] Akalanka Galappaththi, Sarah Nadi, and Christoph Treude. 2024. An Empirical Study of API Misuses of Data-Centric Libraries. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement* (Barcelona, Spain) (ESEM '24). Association for Computing Machinery, New York, NY, USA, 245–256. doi:10.1145/3674805.3686685
- [19] Lee Harrison, Hayawardh Vijayakumar, Rohan Padhye, Koushik Sen, and Michael Grace. 2020. PARTEMU: Enabling Dynamic Analysis of Real-World TrustZone Software Using Emulation. In *29th USENIX Security Symposium* (USENIX Security 20). USENIX Association, 789–806.
- [20] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 124 (May 2025), 30 pages. doi:10.1145/3712003
- [21] Patrick Jaurnig, Ahmad-Reza Sadeghi, and Emmanuel Stappf. 2020. Trusted Execution Environments: Properties, Applications, and Challenges. *IEEE Security & Privacy* 18, 2 (2020), 56–60.
- [22] Mustakimur Rahman Khandaker, Yueqiang Cheng, Zhi Wang, and Tao Wei. 2020. COIN Attacks: On Insecurity of Enclave Untrusted Interfaces in SGX. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '20). Association for Computing Machinery, New York, NY, USA, 971–985.
- [23] Wenhao Li, Yubin Xia, Long Lu, Haibo Chen, and Binyu Zang. 2019. TEEv: virtualizing trusted execution environments on mobile platforms. In *Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments* (Providence, RI, USA) (VEE 2019). Association for Computing Machinery, New York, NY, USA, 2–16. doi:10.1145/3313808.3313810
- [24] Xiaoguo Li, Bowen Zhao, Guomin Yang, Tao Xiang, Jian Weng, and Robert H. Deng. 2023. A Survey of Secure Computation Using Trusted Execution Environments. arXiv:2302.12150 [cs.CR] <https://arxiv.org/abs/2302.12150>
- [25] David Lo. 2023. Trustworthy and Synergistic Artificial Intelligence for Software Engineering: Vision and Roadmaps. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. 69–85. doi:10.1109/ICSE-FoSE59343.2023.00010
- [26] Chengyan Ma, Ruidong Han, Jieke Shi, Ye Liu, Yuqing Niu, Di Lu, Chuang Tian, Jianfeng Ma, Debin Gao, and David Lo. 2025. DITING: A Static Analyzer for Identifying Bad Partitioning Issues in TEE Applications. arXiv:2502.15281 [cs.CR] <https://arxiv.org/abs/2502.15281>
- [27] Sanoop Malliserry and Yu-Sung Wu. 2023. Demystify the Fuzzing Methods: A Comprehensive Survey. *ACM Comput. Surv.* 56, 3, Article 71 (Oct. 2023), 38 pages. doi:10.1145/3623375
- [28] Ruijie Meng, Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. 2025. AFLNet Five Years Later: On Coverage-Guided Protocol Fuzzing. *IEEE Transactions on Software Engineering* 51, 4 (2025), 960–974. doi:10.1109/TSE.2025.3535925
- [29] Zahra Mousavi, Chadni Islam, Muhammad Ali Babar, Alsharif Abudbba, and Kristen Moore. 2025. Detecting Misuse of Security APIs: A Systematic Review. *ACM Comput. Surv.* 57, 12, Article 303 (July 2025), 39 pages. doi:10.1145/3735968
- [30] Sarah Nadi, Stefan Krüger, Mira Mezini, and Eric Bodden. 2016. Jumping through hoops: why do Java developers struggle with cryptography APIs?. In *Proceedings of the 38th International Conference on Software Engineering* (Austin, Texas) (ICSE '16). Association for Computing Machinery, New York, NY, USA, 935–946. doi:10.1145/2884781.2884790

- [31] Olivier Nourry, Yutaro Kashiwa, Bin Lin, Gabriele Bavota, Michele Lanza, and Yasutaka Kamei. 2023. The Human Side of Fuzzing: Challenges Faced by Developers during Fuzzing Activities. *ACM Trans. Softw. Eng. Methodol.* 33, 1, Article 14 (Nov. 2023), 26 pages. doi:10.1145/3611668
- [32] Arm Limited (or its affiliates). 2025. ARM TrustZone Technology. <https://developer.arm.com/documentation/100690/0200/ARM-TrustZone-technology>. [Accessed 05-11-2025].
- [33] Mohamed Sabt, Mohammed Achemlal, and Abdelmadjid Bouabdallah. 2015. Trusted Execution Environment: What It is, and What It is Not. In *2015 IEEE Trustcom/BigDataSE/ISPA*, Vol. 1. 57–64. doi:10.1109/Trustcom.2015.357
- [34] Jieke Shi, Zhou Yang, and David Lo. 2025. Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 137 (May 2025), 22 pages. doi:10.1145/3708525
- [35] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Andrew Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. 2016. SOK: (State of) The Art of War: Offensive Techniques in Binary Analysis. In *2016 IEEE Symposium on Security and Privacy (SP)*. 138–157. doi:10.1109/SP.2016.17
- [36] Bo Yang, Kang Yang, Zhenfeng Zhang, Yu Qin, and Dengguo Feng. 2016. AEP-M: Practical Anonymous E-Payment for Mobile Devices Using ARM TrustZone and Divisible E-Cash. In *Information Security*, Matt Bishop and Anderson C A Nascimento (Eds.). Springer International Publishing, Cham, 130–146.
- [37] Joobeom Yun, Fayozbek Rustamov, Juhwan Kim, and Youngjoo Shin. 2022. Fuzzing of Embedded Systems: A Survey. *ACM Comput. Surv.* 55, 7, Article 137 (Dec. 2022), 33 pages. doi:10.1145/3538644